Universidad / Facultad (completar)

Carrera de Ingeniería (completar)

Informe Técnico: Resiliencia del Estado en Android con Jetpack Compose

Aplicación EcoTrip 2026

Examen Práctico RdA-2 · Grupo 4

James Argüello

Wismar Benítez

Carlos Galarza

Mateo Morales

Ángel Loor

Asignatura (completar)

Docente (completar)

Octubre de 2026

Introducción

En Android, la interfaz de usuario vive en componentes cuyo ciclo de vida no controla el desarrollador. Una rotación de pantalla destruye y recrea la *Activity*; la presión de memoria puede terminar el proceso completo mientras la aplicación está en segundo plano. Cuando el estado reside únicamente en variables de la UI, cualquiera de estos eventos borra lo que el usuario había escrito. A este fenómeno se le denomina volatilidad del estado.

EcoTrip 2026 es una aplicación de planificación de viajes sostenibles que sirve como caso de estudio. Su formulario captura destino, días de duración, modalidad grupal, nombre del viajero, medio de transporte y preferencia de baja huella de carbono. El objetivo es que cada dato sobreviva al nivel de persistencia que le corresponde: memoria, proceso o disco. Este informe describe la arquitectura adoptada, las pruebas de resiliencia ejecutadas en el emulador API 36 (Android 16) y la justificación técnica de la navegación fuertemente tipada.

Arquitectura implementada

La aplicación sigue el patrón de flujo unidireccional de datos (UDF) en tres capas. El código está escrito al 100 % con Jetpack Compose y Material Design 3, sin vistas XML ni *findViewById*. El proyecto usa Java 21, *compileSdk* y *targetSdk* 36, y centraliza las dependencias en el catálogo *libs.versions.toml*.

Componentes y responsabilidades

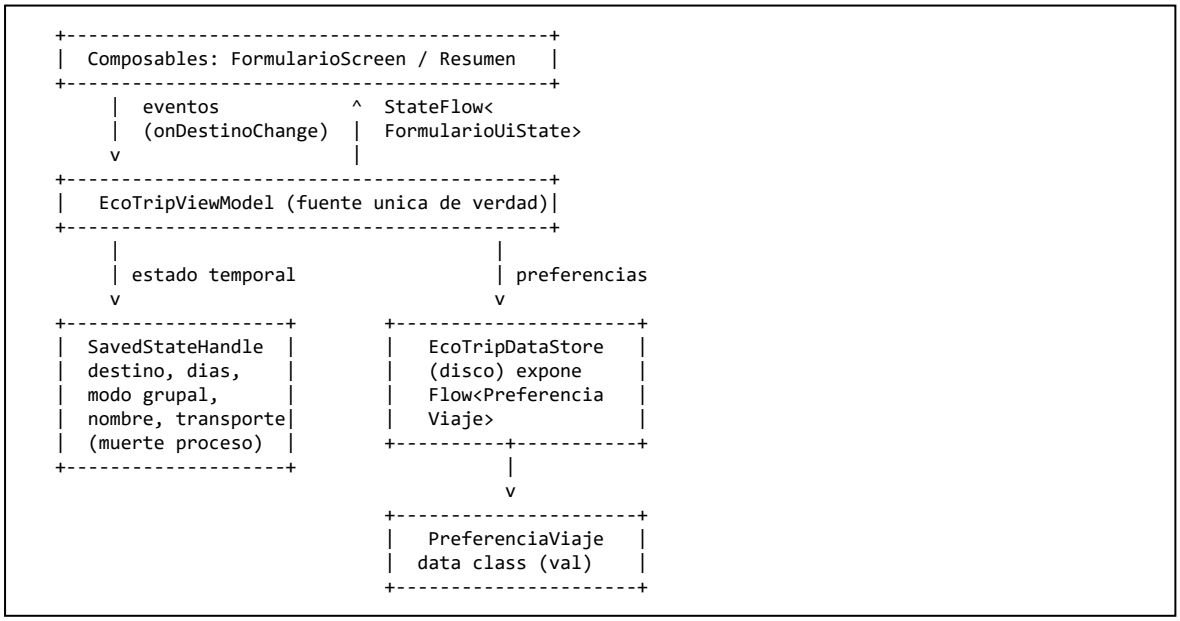
Tabla 1

Niveles de persistencia por componente

Componente	Rol	Sobrevive a rotación	Sobrevive a muerte de proceso	Sobrevive a cierre total
PreferenciaViaje (data class)	Modelo inmutable (*val*) de las preferencias globales	—	—	—
ViewModel (*StateFlow*)	Fuente única de verdad de la UI	Sí	No	No
SavedStateHandle	Destino, días y modo grupal	Sí	Sí	No
DataStore (Preferences)	Nombre, transporte y baja huella	Sí	Sí	Sí

Diagrama conceptual

Figura 1. Interacción entre PreferenciaViaje, ViewModel, SavedStateHandle y DataStore.



La UI (*FormularioScreen*) no guarda lógica: recibe un *FormularioUiState* y emite eventos, aplicando *state hoisting*. El ViewModel actualiza el estado de forma atómica con *MutableStateFlow.update* y, para los tres campos temporales, escribe además en el *SavedStateHandle*, que el sistema serializa en un *Bundle* antes de matar el proceso. Al reconstruirse, el ViewModel lee esos valores como estado inicial. Los datos que deben ser permanentes pasan por *EcoTripDataStore*, que expone un *Flow<PreferenciaViaje>* y escribe con una corrutina (*edit*) sin bloquear el hilo principal. DataStore se prefiere sobre SharedPreferences por su API asíncrona basada en corrutinas, su consistencia transaccional y el manejo explícito de errores de E/S.

La validación se realiza en el ViewModel: los días se interpretan con *toIntOrNull()* y se rechazan valores vacíos, no numéricos o menores o iguales a cero; un destino en blanco también genera error. La navegación al resumen solo ocurre si ambos campos son válidos.

Repositorio

Código fuente disponible públicamente en: https://github.com/JArguelloC/Examen_Practico_RdA2_Grupo4

Evidencias de pruebas de resiliencia

Las pruebas se ejecutaron en el emulador Pixel 8 con Android 16 (API 36). Además de las pruebas manuales, el proyecto incluye pruebas instrumentadas (*ResilienciaTest*) que recrean la Activity y verifican la restauración del *SavedStateHandle*. Para la muerte de proceso se activó «No conservar actividades», se envió la app a segundo plano y se terminó el proceso con *adb shell am kill*: el identificador de proceso cambió (7008 → 7174), lo que confirma que el proceso fue recreado y no solo la Activity. Para el Color Dinámico se aplicó un fondo de pantalla naranja desde el selector del sistema y se reabrió la app: la paleta pasó de azul a naranja sin cambios de código.

Durante la primera ejecución de la prueba de muerte de proceso se detectó un defecto: destino, días y modo grupal se restauraban, pero el nombre, el transporte y la bandera de baja huella volvían a sus valores por defecto, porque solo se guardaban en DataStore al pulsar el botón de guardar. Se corrigió registrando también esos tres campos en el *SavedStateHandle* y dándoles prioridad sobre el valor del disco al iniciar. Las capturas de este informe corresponden a la versión corregida.

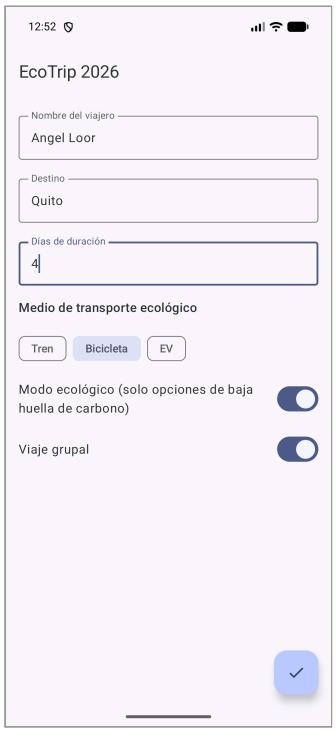


Figura 2. Formulario de viaje con datos válidos (modo vertical, emulador API 36).



Figura 3. Prueba de rotación: el formulario conserva los datos en modo horizontal.

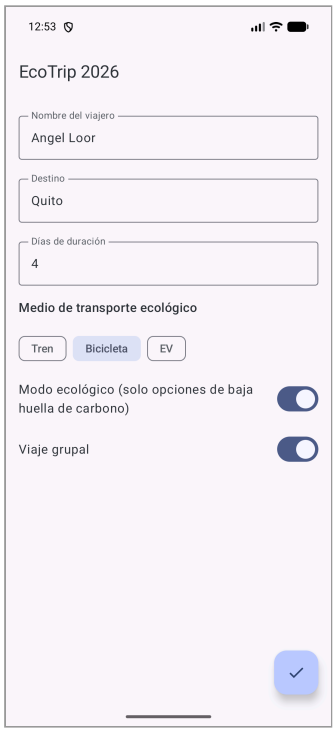


Figura 4. Muerte de proceso, antes: formulario lleno con «No conservar actividades» activo.

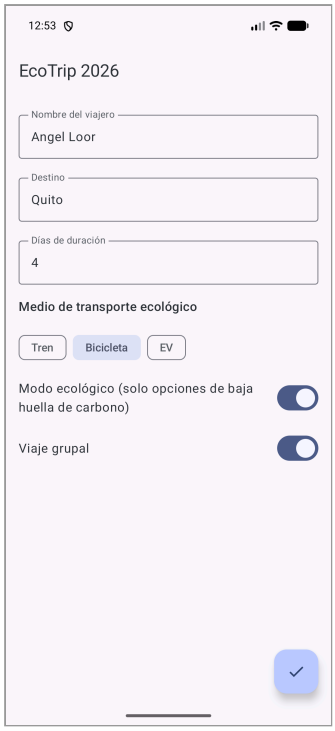


Figura 5. Muerte de proceso, después: el proceso fue terminado (am kill) y todos los campos se restauraron.

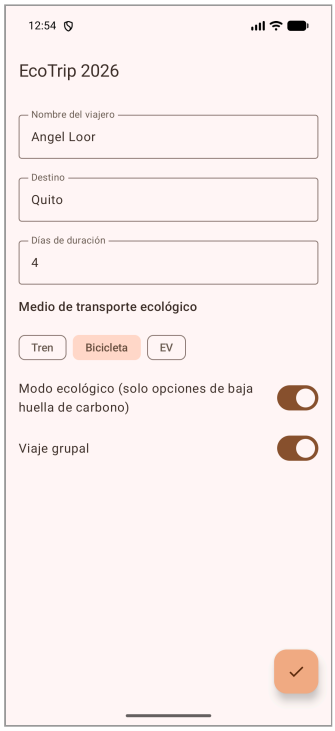


Figura 6. Formulario con Color Dinámico tras cambiar el fondo de pantalla a naranja.



Figura 7. Pantalla de Resumen de Ruta con Color Dinámico tras cambiar el fondo de pantalla.

Tabla 2

Resultados de las pruebas

Prueba	**Procedimiento**	**Resultado**
Rotación	Llenar formulario y rotar vertical ↔ horizontal	OK. Los cinco campos conservan su valor
Muerte de proceso	Activar «No conservar actividades», enviar la app a segundo plano, *am kill* y regresar	OK tras la corrección (falló antes con 3 de 6 campos)
Material You	Cambiar el fondo de pantalla a naranja y reabrir la app	OK. La paleta de la UI cambió a tonos naranja
Instrumentadas	*./gradlew connectedDebugAndroidTest*	OK. 6 de 6 pruebas aprobadas

Navegación fuertemente tipada frente a Strings y Bundles

En el esquema clásico, cada destino se identificaba con una cadena (*"resumen/{destino}/{dias}"*) y los argumentos viajaban como texto o dentro de un *Bundle*. Ese diseño traslada al tiempo de ejecución errores que el compilador no puede ver:

1. **Rutas mal escritas.** Un error tipográfico en la cadena de una ruta solo se descubre al navegar, con un *IllegalArgumentException* por destino inexistente.

2. **Claves de argumento inconsistentes.** Quien escribe la clave *"dias"* y quien lee *"duracion"* obtiene *null* o un valor por defecto, sin aviso del compilador.

3. **Tipos no verificados.** Los argumentos de ruta llegan como *String* y requieren conversiones manuales; un *"abc"* donde se espera un entero produce *NumberFormatException* o un *ClassCastException* al leer el *Bundle*.

4. **Argumentos obligatorios ausentes.** Nada impide omitir un parámetro al construir la ruta.

Con Navigation Compose 2.8 las rutas son clases *@Serializable*. En EcoTrip, *ResumenRoute* es una *data class* con seis campos tipados (*diasDuracion: Int*, *soloBajaHuella: Boolean*, etc.). El destino se declara con *composable<ResumenRoute>*, se invoca con *navController.navigate(ResumenRoute(...))* y los argumentos se recuperan con *backStackEntry.toRoute<ResumenRoute>()*. La serialización y deserialización las genera el plugin de *kotlinx.serialization* en tiempo de compilación, de modo que:

• No existen cadenas de ruta que escribir a mano: el tipo es el identificador.

• Faltar un argumento o pasar un tipo incorrecto es un error de compilación, no un *crash*.

• Renombrar o añadir un campo se propaga a todos los puntos de uso mediante el compilador y el IDE.

• La conversión de tipos ocurre en un único lugar controlado por la librería.

Es importante precisar el alcance: el tipado fuerte elimina los fallos de estructura, nombre y tipo, pero no los de valor. Por eso EcoTrip valida los datos en el ViewModel antes de navegar. Para la higiene del *back stack*, el retorno desde el resumen usa

popUpTo<FormularioRoute> con *launchSingleTop = true*, evitando formularios duplicados en la pila.

Conclusiones y recomendaciones

Los resultados de las pruebas de resiliencia (Tabla 2) respaldan la estrategia de separar el estado por su ciclo de vida: ViewModel para la recomposición y rotación, *SavedStateHandle* para la muerte de proceso y DataStore para la persistencia permanente. La navegación tipada reduce la superficie de errores en tiempo de ejecución y concentra los restantes en la validación de valores.

Recomendaciones de ingeniería:

1. Registrar en *SavedStateHandle* todo campo de entrada, aunque también tenga destino en DataStore. La prueba de muerte de proceso demostró que lo que solo vive en memoria o en un almacenamiento que se escribe «al guardar» se pierde antes de guardar. Esta recomendación ya fue aplicada en la versión entregada.

2. Mantener Espresso actualizado: con la versión 3.6.1 las pruebas instrumentadas fallan en Android 16 por la eliminación de *InputManager.getInstance*; se corrigió subiendo a 3.7.0 en el catálogo de versiones.

3. Incorporar pruebas unitarias del ViewModel con un *DataStore* inyectable mediante una interfaz, para ejecutarlas sin dispositivo.

4. Incluir la prueba de «No conservar actividades» en la lista de verificación de cada entrega, ya que es la forma más barata de detectar estado perdido.

5. Pasar a la pantalla de resumen solo un identificador o recuperar los datos desde una única fuente, en lugar de copiar seis campos en la ruta, si el modelo crece.

Referencias

Android Developers. (s. f.). *Save UI states*. Google. https://developer.android.com/topic/libraries/architecture/saving-states

Android Developers. (s. f.). *Type safety in Kotlin DSL and Navigation Compose*. Google. https://developer.android.com/guide/navigation/design/type-safety

Android Developers. (s. f.). *DataStore*. Google. https://developer.android.com/topic/libraries/architecture/datastore

Android Developers. (s. f.). *Dynamic color* (Material Design 3). Google. https://developer.android.com/develop/ui/views/theming/dynamic-colors

Anexo. Código fuente de la aplicación

app/build.gradle.kts

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.kotlin.serialization)
}

android {
    namespace = "com.example.myapplication"
    compileSdk = 36

    defaultConfig {
        applicationId = "com.example.myapplication"
        minSdk = 26
        targetSdk = 36
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_21
        targetCompatibility = JavaVersion.VERSION_21
    }
    kotlinOptions {
        jvmTarget = "21"
    }
    buildFeatures {
        compose = true
    }
}

dependencies {
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    implementation(libs.androidx.activity.compose)
    implementation(platform(libs.androidx.compose.bom))
    implementation(libs.androidx.ui)
    implementation(libs.androidx.ui.graphics)
    implementation(libs.androidx.ui.tooling.preview)
    implementation(libs.androidx.material3)
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(platform(libs.androidx.compose.bom))
    androidTestImplementation(libs.androidx.ui.test.junit4)
    debugImplementation(libs.androidx.ui.tooling)
    debugImplementation(libs.androidx.ui.test.manifest)
    implementation(libs.androidx.lifecycle.viewmodel.compose)
    implementation(libs.androidx.navigation.compose)
    implementation(libs.kotlinx.serialization.json)
    implementation(libs.androidx.datastore.preferences)
}
```

gradle/libs.versions.toml

```
[versions]
agp = "8.7.0"
kotlin = "2.0.0"
coreKtx = "1.13.1"
junit = "4.13.2"
junitVersion = "1.2.1"
espressoCore = "3.7.0"
lifecycleRuntimeKtx = "2.8.6"
activityCompose = "1.9.3"
composeBom = "2024.09.03"
# Nuevas versiones requeridas para el Examen RDA-2
navigationCompose = "2.8.3"
kotlinxSerialization = "1.6.3"
datastorePreferences = "1.1.1"

[libraries]
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
junit = { group = "junit", name = "junit", version.ref = "junit" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref =
"espressoCore" }
androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx",
version.ref = "lifecycleRuntimeKtx" }
androidx-lifecycle-viewmodel-compose = { group = "androidx.lifecycle", name = "lifecycle-viewmodel-
compose", version.ref = "lifecycleRuntimeKtx" }
androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref =
"activityCompose" }
androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }
androidx-ui = { group = "androidx.compose.ui", name = "ui" }
androidx-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
androidx-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
androidx-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
androidx-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
androidx-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
androidx-material3 = { group = "androidx.compose.material3", name = "material3" }

# Rutas Tipadas y Serialización
androidx-navigation-compose = { group = "androidx.navigation", name = "navigation-compose", version.ref
= "navigationCompose" }
kotlinx-serialization-json = { group = "org.jetbrains.kotlinx", name = "kotlinx-serialization-json",
version.ref = "kotlinxSerialization" }

# Persistencia en Disco
androidx-datastore-preferences = { group = "androidx.datastore", name = "datastore-preferences",
version.ref = "datastorePreferences" }

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
kotlin-android = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }
kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
kotlin-serialization = { id = "org.jetbrains.kotlin.plugin.serialization", version.ref = "kotlin" }
```

app/src/main/java/com/example/myapplication/MainActivity.kt

```
package com.example.myapplication

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.lifecycle.viewmodel.compose.viewModel
import com.example.myapplication.navigation.EcoTripNavHost
import com.example.myapplication.ui.theme.MyApplicationTheme
import com.example.myapplication.ui.viewmodel.EcoTripViewModel

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyApplicationTheme {
                val ecoTripViewModel: EcoTripViewModel = viewModel(
```

```
                    factory = EcoTripViewModel.factory(application)
                )
                EcoTripNavHost(viewModel = ecoTripViewModel)
            }
        }
    }
}
```

app/src/main/java/com/example/myapplication/data/model/PreferenciaViaje.kt

```
package com.example.myapplication.data.model

import kotlinx.serialization.Serializable

enum class TipoTransporte {
    TREN,
    BICICLETA,
    VEHICULO_ELECTRICO
}

@Serializable
data class PreferenciaViaje(
    val nombreViajero: String = "",
    val transporteSeleccionado: TipoTransporte = TipoTransporte.TREN,
    val soloBajaHuella: Boolean = false
)
```

app/src/main/java/com/example/myapplication/data/local/EcoTripDataStore.kt

```
package com.example.myapplication.data.local

import android.content.Context
import androidx.datastore.core.DataStore
import androidx.datastore.preferences.core.Preferences
import androidx.datastore.preferences.core.booleanPreferencesKey
import androidx.datastore.preferences.core.edit
import androidx.datastore.preferences.core.emptyPreferences
import androidx.datastore.preferences.core.stringPreferencesKey
import androidx.datastore.preferences.preferencesDataStore
import com.example.myapplication.data.model.PreferenciaViaje
import com.example.myapplication.data.model.TipoTransporte
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.catch
import kotlinx.coroutines.flow.map
import java.io.IOException

val Context.dataStore: DataStore<Preferences> by preferencesDataStore(name = "ecotrip_preferences")

class EcoTripDataStore(private val context: Context) {

    companion object {
        val KEY_NOMBRE_VIAJERO = stringPreferencesKey("key_nombre_viajero")
        val KEY_TIPO_TRANSPORTE = stringPreferencesKey("key_tipo_transporte")
        val KEY_SOLO_BAJA_HUELLA = booleanPreferencesKey("key_solo_baja_huella")
    }

    val preferenciaViajeFlow: Flow<PreferenciaViaje> = context.dataStore.data
        .catch { exception ->
            if (exception is IOException) {
                emit(emptyPreferences())
            } else {
                throw exception
            }
        }
        .map { preferences ->
            val nombre = preferences[KEY_NOMBRE_VIAJERO] ?: ""
            val transporteStr = preferences[KEY_TIPO_TRANSPORTE] ?: TipoTransporte.TREN.name
            val transporte = try {
                TipoTransporte.valueOf(transporteStr)
            } catch (e: IllegalArgumentException) {
                TipoTransporte.TREN
            }
```

```
            val bajaHuella = preferences[KEY_SOLO_BAJA_HUELLA] ?: false

            PreferenciaViaje(
                nombreViajero = nombre,
                transporteSeleccionado = transporte,
                soloBajaHuella = bajaHuella
            )
        }

    suspend fun guardarPreferenciasGlobales(preferencia: PreferenciaViaje) {
        context.dataStore.edit { preferences ->
            preferences[KEY_NOMBRE_VIAJERO] = preferencia.nombreViajero
            preferences[KEY_TIPO_TRANSPORTE] = preferencia.transporteSeleccionado.name
            preferences[KEY_SOLO_BAJA_HUELLA] = preferencia.soloBajaHuella
        }
    }
}
```

app/src/main/java/com/example/myapplication/ui/state/FormularioUiState.kt

```
package com.example.myapplication.ui.state

import com.example.myapplication.data.model.TipoTransporte

data class FormularioUiState(
    // Persistencia permanente (DataStore)
    val nombreViajero: String = "",
    val transporteSeleccionado: TipoTransporte = TipoTransporte.TREN,
    val soloBajaHuella: Boolean = false,

    // Persistencia de proceso (SavedStateHandle)
    val destino: String = "",
    val diasDuracionTexto: String = "",
    val esViajeGrupal: Boolean = false,

    // Feedback de validación
    val errorDestino: String? = null,
    val errorDias: String? = null
)
```

app/src/main/java/com/example/myapplication/ui/viewmodel/EcoTripViewModel.kt

```
package com.example.myapplication.ui.viewmodel

import android.app.Application
import androidx.lifecycle.SavedStateHandle
import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.viewModelScope
import androidx.lifecycle.createSavedStateHandle
import androidx.lifecycle.viewmodel.initializer
import androidx.lifecycle.viewmodel.viewModelFactory
import com.example.myapplication.data.local.EcoTripDataStore
import com.example.myapplication.data.model.PreferenciaViaje
import com.example.myapplication.data.model.TipoTransporte
import com.example.myapplication.ui.state.FormularioUiState
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch

class EcoTripViewModel(
    private val savedStateHandle: SavedStateHandle,
    private val dataStore: EcoTripDataStore
) : ViewModel() {

    companion object {
        const val KEY_DESTINO = "destino"
        const val KEY_DIAS = "dias"
        const val KEY_ES_GRUPAL = "esGrupal"
        const val KEY_NOMBRE = "nombreViajero"
```

```
        const val KEY_TRANSPORTE = "transporte"
        const val KEY_BAJA_HUELLA = "soloBajaHuella"

        fun factory(application: Application): ViewModelProvider.Factory = viewModelFactory {
            initializer {
                EcoTripViewModel(
                    savedStateHandle = createSavedStateHandle(),
                    dataStore = EcoTripDataStore(application.applicationContext)
                )
            }
        }
    }

    // Estado inicial restaurado desde SavedStateHandle (sobrevive a process death)
    private val _uiState = MutableStateFlow(
        FormularioUiState(
            destino = savedStateHandle.get<String>(KEY_DESTINO) ?: "",
            diasDuracionTexto = savedStateHandle.get<String>(KEY_DIAS) ?: "",
            esViajeGrupal = savedStateHandle.get<Boolean>(KEY_ES_GRUPAL) ?: false,
            nombreViajero = savedStateHandle.get<String>(KEY_NOMBRE) ?: "",
            transporteSeleccionado = savedStateHandle.get<String>(KEY_TRANSPORTE)
                ?.let { runCatching { TipoTransporte.valueOf(it) }.getOrNull() }
                ?: TipoTransporte.TREN,
            soloBajaHuella = savedStateHandle.get<Boolean>(KEY_BAJA_HUELLA) ?: false
        )
    )
    val uiState: StateFlow<FormularioUiState> = _uiState.asStateFlow()

    init {
        cargarPreferencias()
    }

    fun onDestinoChange(nuevoDestino: String) {
        savedStateHandle[KEY_DESTINO] = nuevoDestino
        _uiState.update {
            it.copy(
                destino = nuevoDestino,
                errorDestino = if (nuevoDestino.isBlank()) "El destino es obligatorio" else null
            )
        }
    }

    fun onDiasChange(nuevosDias: String) {
        val texto = nuevosDias.trim()
        savedStateHandle[KEY_DIAS] = texto
        val dias = texto.toIntOrNull() ?: 0
        val error = when {
            texto.isEmpty() -> "Ingresa los días de duración"
            texto.toIntOrNull() == null -> "Ingresa un número válido"
            dias <= 0 -> "Debe ser mayor a 0"
            else -> null
        }
        _uiState.update { it.copy(diasDuracionTexto = texto, errorDias = error) }
    }

    fun onModoGrupalChange(esGrupal: Boolean) {
        savedStateHandle[KEY_ES_GRUPAL] = esGrupal
        _uiState.update { it.copy(esViajeGrupal = esGrupal) }
    }

    fun onNombreChange(nuevoNombre: String) {
        savedStateHandle[KEY_NOMBRE] = nuevoNombre
        _uiState.update { it.copy(nombreViajero = nuevoNombre) }
    }

    fun onTransporteChange(transporte: TipoTransporte) {
        savedStateHandle[KEY_TRANSPORTE] = transporte.name
        _uiState.update { it.copy(transporteSeleccionado = transporte) }
    }

    fun onSoloBajaHuellaChange(soloBajaHuella: Boolean) {
        savedStateHandle[KEY_BAJA_HUELLA] = soloBajaHuella
        _uiState.update { it.copy(soloBajaHuella = soloBajaHuella) }
    }
```

```
    fun guardarPreferencias() {
        val estado = _uiState.value
        viewModelScope.launch {
            dataStore.guardarPreferenciasGlobales(
                PreferenciaViaje(
                    nombreViajero = estado.nombreViajero,
                    transporteSeleccionado = estado.transporteSeleccionado,
                    soloBajaHuella = estado.soloBajaHuella
                )
            )
        }
    }

    /** Sincroniza el estado de UI con lo guardado en DataStore. */
    private fun cargarPreferencias() {
        viewModelScope.launch {
            dataStore.preferenciaViajeFlow.collect { pref ->
                _uiState.update {
                    // Lo que el usuario ya escribió (SavedStateHandle) prevalece sobre el disco.
                    it.copy(
                        nombreViajero = if (savedStateHandle.contains(KEY_NOMBRE)) it.nombreViajero else
pref.nombreViajero,
                        transporteSeleccionado = if (savedStateHandle.contains(KEY_TRANSPORTE))
it.transporteSeleccionado else pref.transporteSeleccionado,
                        soloBajaHuella = if (savedStateHandle.contains(KEY_BAJA_HUELLA))
it.soloBajaHuella else pref.soloBajaHuella
                    )
                }
            }
        }
    }
}
```

app/src/main/java/com/example/myapplication/navigation/Routes.kt

```
package com.example.myapplication.navigation

import kotlinx.serialization.Serializable

@Serializable
object FormularioRoute

@Serializable
data class ResumenRoute(
    val nombreViajero: String,
    val transporte: String,
    val soloBajaHuella: Boolean,
    val destino: String,
    val diasDuracion: Int,
    val esViajeGrupal: Boolean
)
```

app/src/main/java/com/example/myapplication/navigation/EcoTripNavHost.kt

```
package com.example.myapplication.navigation

import androidx.compose.runtime.Composable
import androidx.navigation.NavHostController
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import androidx.navigation.toRoute
import com.example.myapplication.ui.screens.FormularioScreen
import com.example.myapplication.ui.screens.ResumenScreen
import com.example.myapplication.ui.viewmodel.EcoTripViewModel

@Composable
fun EcoTripNavHost(
    viewModel: EcoTripViewModel,
    navController: NavHostController = rememberNavController()
) {
```

```
    NavHost(
        navController = navController,
        startDestination = FormularioRoute
    ) {
        composable<FormularioRoute> {
            FormularioScreen(
                viewModel = viewModel,
                onNavigateToResumen = {
                    val state = viewModel.uiState.value
                    val diasDuracion = state.diasDuracionTexto.trim().toIntOrNull()

                    if (state.destino.isNotBlank() && diasDuracion != null && diasDuracion > 0) {
                        navController.navigate(
                            ResumenRoute(
                                nombreViajero = state.nombreViajero,
                                transporte = state.transporteSeleccionado.name,
                                soloBajaHuella = state.soloBajaHuella,
                                destino = state.destino.trim(),
                                diasDuracion = diasDuracion,
                                esViajeGrupal = state.esViajeGrupal
                            )
                        )
                    }
                }
            )
        }

        composable<ResumenRoute> { backStackEntry ->
            val route = backStackEntry.toRoute<ResumenRoute>()
            ResumenScreen(
                route = route,
                onVolver = {
                    navController.navigate(FormularioRoute) {
                        popUpTo<FormularioRoute> {
                            inclusive = false
                        }
                        launchSingleTop = true
                    }
                }
            )
        }
    }
}
```

app/src/main/java/com/example/myapplication/ui/screens/FormularioScreen.kt

```
package com.example.myapplication.ui.screens

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Check
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.FilterChip
import androidx.compose.material3.FloatingActionButton
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Switch
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
```

```
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.myapplication.data.model.TipoTransporte
import com.example.myapplication.ui.state.FormularioUiState
import com.example.myapplication.ui.viewmodel.EcoTripViewModel

/** Contenedor con estado: conecta el ViewModel con la UI sin estado. */
@Composable
fun FormularioScreen(
    viewModel: EcoTripViewModel,
    onNavigateToResumen: () -> Unit
) {
    val state by viewModel.uiState.collectAsState()

    FormularioContent(
        state = state,
        onNombreChange = viewModel::onNombreChange,
        onDestinoChange = viewModel::onDestinoChange,
        onDiasChange = viewModel::onDiasChange,
        onTransporteChange = viewModel::onTransporteChange,
        onSoloBajaHuellaChange = viewModel::onSoloBajaHuellaChange,
        onModoGrupalChange = viewModel::onModoGrupalChange,
        onGuardar = {
            // Fuerza la validación de los campos obligatorios aún no tocados.
            viewModel.onDestinoChange(state.destino)
            viewModel.onDiasChange(state.diasDuracionTexto)
            val dias = state.diasDuracionTexto.trim().toIntOrNull()
            if (state.destino.isNotBlank() && dias != null && dias > 0) {
                viewModel.guardarPreferencias()
                onNavigateToResumen()
            }
        }
    )
}

/** UI sin estado: solo recibe el estado inmutable y callbacks. */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun FormularioContent(
    state: FormularioUiState,
    onNombreChange: (String) -> Unit,
    onDestinoChange: (String) -> Unit,
    onDiasChange: (String) -> Unit,
    onTransporteChange: (TipoTransporte) -> Unit,
    onSoloBajaHuellaChange: (Boolean) -> Unit,
    onModoGrupalChange: (Boolean) -> Unit,
    onGuardar: () -> Unit,
    modifier: Modifier = Modifier
) {
    Scaffold(
        modifier = modifier,
        topBar = { TopAppBar(title = { Text("EcoTrip 2026") }) },
        floatingActionButton = {
            FloatingActionButton(onClick = onGuardar) {
                Icon(Icons.Filled.Check, contentDescription = "Guardar y ver resumen")
            }
        }
    ) { innerPadding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding)
                .verticalScroll(rememberScrollState())
                .padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp)
        ) {
            OutlinedTextField(
                value = state.nombreViajero,
                onValueChange = onNombreChange,
                label = { Text("Nombre del viajero") },
                singleLine = true,
                modifier = Modifier.fillMaxWidth()
```

```
            )

            OutlinedTextField(
                value = state.destino,
                onValueChange = onDestinoChange,
                label = { Text("Destino") },
                isError = state.errorDestino != null,
                supportingText = state.errorDestino?.let { { Text(it) } },
                singleLine = true,
                modifier = Modifier.fillMaxWidth()
            )

            OutlinedTextField(
                value = state.diasDuracionTexto,
                onValueChange = onDiasChange,
                label = { Text("Días de duración") },
                isError = state.errorDias != null,
                supportingText = state.errorDias?.let { { Text(it) } },
                keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
                singleLine = true,
                modifier = Modifier.fillMaxWidth()
            )

            Text(
                text = "Medio de transporte ecológico",
                style = MaterialTheme.typography.titleMedium
            )
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.spacedBy(8.dp)
            ) {
                TipoTransporte.entries.forEach { transporte ->
                    FilterChip(
                        selected = state.transporteSeleccionado == transporte,
                        onClick = { onTransporteChange(transporte) },
                        label = { Text(transporte.etiqueta()) }
                    )
                }
            }

            SwitchRow(
                texto = "Modo ecológico (solo opciones de baja huella de carbono)",
                checked = state.soloBajaHuella,
                onCheckedChange = onSoloBajaHuellaChange
            )

            SwitchRow(
                texto = "Viaje grupal",
                checked = state.esViajeGrupal,
                onCheckedChange = onModoGrupalChange
            )
        }
    }
}

@Composable
private fun SwitchRow(
    texto: String,
    checked: Boolean,
    onCheckedChange: (Boolean) -> Unit
) {
    Row(
        modifier = Modifier.fillMaxWidth(),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.SpaceBetween
    ) {
        Text(
            text = texto,
            style = MaterialTheme.typography.bodyLarge,
            modifier = Modifier
                .weight(1f)
                .padding(end = 16.dp)
        )
        Switch(checked = checked, onCheckedChange = onCheckedChange)
```

```
    }
}

private fun TipoTransporte.etiqueta(): String = when (this) {
    TipoTransporte.TREN -> "Tren"
    TipoTransporte.BICICLETA -> "Bicicleta"
    TipoTransporte.VEHICULO_ELECTRICO -> "EV"
}

@Preview(showBackground = true)
@Composable
private fun FormularioContentPreview() {
    FormularioContent(
        state = FormularioUiState(destino = "Quito", diasDuracionTexto = "3"),
        onNombreChange = {},
        onDestinoChange = {},
        onDiasChange = {},
        onTransporteChange = {},
        onSoloBajaHuellaChange = {},
        onModoGrupalChange = {},
        onGuardar = {}
    )
}
```

app/src/main/java/com/example/myapplication/ui/screens/ResumenScreen.kt

```
package com.example.myapplication.ui.screens

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.BoxWithConstraints
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material.icons.filled.Info
import androidx.compose.material.icons.filled.Person
import androidx.compose.material.icons.filled.Place
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.myapplication.data.model.TipoTransporte
import com.example.myapplication.navigation.ResumenRoute
import com.example.myapplication.ui.theme.MyApplicationTheme

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ResumenScreen(
    route: ResumenRoute,
    onVolver: () -> Unit,
    modifier: Modifier = Modifier
) {
    Scaffold(
        modifier = modifier,
```

```
        containerColor = MaterialTheme.colorScheme.background,
        topBar = {
            TopAppBar(
                title = { Text("Resumen de ruta") },
                navigationIcon = {
                    IconButton(onClick = onVolver) {
                        Icon(
                            imageVector = Icons.AutoMirrored.Filled.ArrowBack,
                            contentDescription = "Volver al formulario"
                        )
                    }
                }
            )
        }
    ) { innerPadding ->
        BoxWithConstraints(
            modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding)
                .verticalScroll(rememberScrollState())
                .padding(16.dp)
        ) {
            val wideLayout = maxWidth >= 600.dp
            Column(
                verticalArrangement = Arrangement.spacedBy(16.dp),
                modifier = Modifier.fillMaxWidth()
            ) {
                Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
                    Text(
                        text = "Tu viaje está listo",
                        style = MaterialTheme.typography.headlineMedium,
                        color = MaterialTheme.colorScheme.onSurface,
                        fontWeight = FontWeight.Bold
                    )
                    Text(
                        text = "Revisa los detalles de tu próxima aventura.",
                        style = MaterialTheme.typography.bodyLarge,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                }

                DestinationCard(
                    destino = route.destino,
                    diasDuracion = route.diasDuracion
                )

                if (wideLayout) {
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.spacedBy(16.dp),
                        verticalAlignment = Alignment.Top
                    ) {
                        TravelerCard(
                            nombreViajero = route.nombreViajero,
                            esViajeGrupal = route.esViajeGrupal,
                            modifier = Modifier.weight(1f)
                        )
                        TransportCard(
                            transporte = route.transporte,
                            soloBajaHuella = route.soloBajaHuella,
                            modifier = Modifier.weight(1f)
                        )
                    }
                } else {
                    TravelerCard(
                        nombreViajero = route.nombreViajero,
                        esViajeGrupal = route.esViajeGrupal
                    )
                    TransportCard(
                        transporte = route.transporte,
                        soloBajaHuella = route.soloBajaHuella
                    )
                }
            }
```

```
        }
    }
}

@Composable
private fun DestinationCard(
    destino: String,
    diasDuracion: Int,
    modifier: Modifier = Modifier
) {
    Card(
        modifier = modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.primaryContainer,
            contentColor = MaterialTheme.colorScheme.onPrimaryContainer
        ),
        shape = RoundedCornerShape(24.dp)
    ) {
        Row(
            modifier = Modifier.padding(20.dp),
            horizontalArrangement = Arrangement.spacedBy(16.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Icon(
                imageVector = Icons.Filled.Place,
                contentDescription = null
            )
            Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
                Text(
                    text = destino.ifBlank { "Destino sin especificar" },
                    style = MaterialTheme.typography.headlineSmall,
                    fontWeight = FontWeight.SemiBold
                )
                Text(
                    text = "$diasDuracion ${if (diasDuracion == 1) "día" else "días"}",
                    style = MaterialTheme.typography.titleMedium
                )
            }
        }
    }
}

@Composable
private fun TravelerCard(
    nombreViajero: String,
    esViajeGrupal: Boolean,
    modifier: Modifier = Modifier
) {
    DetailCard(
        title = "Viajero",
        icon = Icons.Filled.Person,
        modifier = modifier
    ) {
        DetailLine(
            label = "Nombre",
            value = nombreViajero.ifBlank { "No especificado" }
        )
        StatusIndicator(
            label = "Modalidad",
            value = if (esViajeGrupal) "Grupal" else "Individual"
        )
    }
}

@Composable
private fun TransportCard(
    transporte: String,
    soloBajaHuella: Boolean,
    modifier: Modifier = Modifier
) {
    DetailCard(
        title = "Movilidad sostenible",
        icon = Icons.Filled.Info,
        modifier = modifier
```

```
    ) {
        DetailLine(
            label = "Transporte",
            value = transporte.toTransportLabel()
        )
        StatusIndicator(
            label = "Rutas de baja huella",
            value = if (soloBajaHuella) "Activado" else "Desactivado"
        )
    }
}

@Composable
private fun DetailCard(
    title: String,
    icon: ImageVector,
    modifier: Modifier = Modifier,
    content: @Composable () -> Unit
) {
    Card(
        modifier = modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surface
        ),
        border = CardDefaults.outlinedCardBorder(),
        shape = RoundedCornerShape(20.dp)
    ) {
        Column(
            modifier = Modifier.padding(20.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp)
        ) {
            Row(
                horizontalArrangement = Arrangement.spacedBy(12.dp),
                verticalAlignment = Alignment.CenterVertically
            ) {
                Icon(
                    imageVector = icon,
                    contentDescription = null,
                    tint = MaterialTheme.colorScheme.primary
                )
                Text(
                    text = title,
                    style = MaterialTheme.typography.titleLarge,
                    color = MaterialTheme.colorScheme.onSurface,
                    fontWeight = FontWeight.SemiBold
                )
            }
            content()
        }
    }
}

@Composable
private fun DetailLine(
    label: String,
    value: String
) {
    Column(verticalArrangement = Arrangement.spacedBy(2.dp)) {
        Text(
            text = label,
            style = MaterialTheme.typography.labelLarge,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
        Text(
            text = value,
            style = MaterialTheme.typography.bodyLarge,
            color = MaterialTheme.colorScheme.onSurface
        )
    }
}

@Composable
private fun StatusIndicator(
    label: String,
```

```
    value: String
) {
    Surface(
        color = MaterialTheme.colorScheme.secondaryContainer,
        contentColor = MaterialTheme.colorScheme.onSecondaryContainer,
        shape = RoundedCornerShape(12.dp)
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(horizontal = 12.dp, vertical = 10.dp),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Text(
                text = label,
                style = MaterialTheme.typography.bodyMedium,
                modifier = Modifier.weight(1f)
            )
            Text(
                text = value,
                style = MaterialTheme.typography.labelLarge,
                fontWeight = FontWeight.SemiBold
            )
        }
    }
}

private fun String.toTransportLabel(): String = when (this) {
    TipoTransporte.TREN.name -> "Tren"
    TipoTransporte.BICICLETA.name -> "Bicicleta"
    TipoTransporte.VEHICULO_ELECTRICO.name -> "Vehículo eléctrico"
    else -> "No especificado"
}

@Preview(showBackground = true, widthDp = 390, heightDp = 800)
@Composable
private fun ResumenScreenPreview() {
    MyApplicationTheme(dynamicColor = false) {
        ResumenScreen(
            route = ResumenRoute(
                nombreViajero = "Mateo Morales",
                transporte = TipoTransporte.TREN.name,
                soloBajaHuella = true,
                destino = "Parque Nacional Cotopaxi",
                diasDuracion = 3,
                esViajeGrupal = true
            ),
            onVolver = {}
        )
    }
}
```

app/src/androidTest/java/com/example/myapplication/ResilienciaTest.kt

```
package com.example.myapplication

import android.content.Context
import androidx.compose.ui.test.hasSetTextAction
import androidx.compose.ui.test.hasText
import androidx.compose.ui.test.junit4.createAndroidComposeRule
import androidx.compose.ui.test.onNodeWithText
import androidx.compose.ui.test.performTextInput
import androidx.lifecycle.SavedStateHandle
import androidx.test.core.app.ApplicationProvider
import androidx.test.ext.junit.runners.AndroidJUnit4
import com.example.myapplication.data.local.EcoTripDataStore
import com.example.myapplication.data.model.TipoTransporte
import com.example.myapplication.ui.viewmodel.EcoTripViewModel
import org.junit.Assert.assertEquals
import org.junit.Assert.assertFalse
import org.junit.Assert.assertNotNull
import org.junit.Assert.assertNull
```

```
import org.junit.Assert.assertTrue
import org.junit.Rule
import org.junit.Test
import org.junit.runner.RunWith

/**
 * Pruebas de resiliencia (QA).
 *
 * - Rotación: recrea la Activity (igual que girar el dispositivo) y verifica que el formulario conserve
los datos.
 * - Muerte de proceso: simula la restauración entregando al ViewModel un SavedStateHandle con los datos
 *   guardados, que es lo que el sistema reconstruye tras matar el proceso.
 */
@RunWith(AndroidJUnit4::class)
class ResilienciaTest {

    @get:Rule
    val composeRule = createAndroidComposeRule<MainActivity>()

    @Test
    fun rotacion_conservaDestinoYDias() {
        composeRule.onNode(hasText("Destino") and hasSetTextAction()).performTextInput("Quito")
        composeRule.onNode(hasText("Días de duración") and hasSetTextAction()).performTextInput("4")

        composeRule.activityRule.scenario.recreate()

        composeRule.onNodeWithText("Quito").assertExists()
        composeRule.onNodeWithText("4").assertExists()
    }

    @Test
    fun procesoMuerto_viewModelRestauraFormularioDesdeSavedStateHandle() {
        val context = ApplicationProvider.getApplicationContext<Context>()
        val estadoRestaurado = SavedStateHandle(
            mapOf(
                EcoTripViewModel.KEY_DESTINO to "Galápagos",
                EcoTripViewModel.KEY_DIAS to "7",
                EcoTripViewModel.KEY_ES_GRUPAL to true,
                EcoTripViewModel.KEY_NOMBRE to "Ángel",
                EcoTripViewModel.KEY_TRANSPORTE to TipoTransporte.BICICLETA.name,
                EcoTripViewModel.KEY_BAJA_HUELLA to true
            )
        )

        val viewModel = EcoTripViewModel(estadoRestaurado, EcoTripDataStore(context))
        val state = viewModel.uiState.value

        assertEquals("Galápagos", state.destino)
        assertEquals("7", state.diasDuracionTexto)
        assertTrue(state.esViajeGrupal)
        assertEquals("Ángel", state.nombreViajero)
        assertEquals(TipoTransporte.BICICLETA, state.transporteSeleccionado)
        assertTrue(state.soloBajaHuella)
    }

    @Test
    fun savedStateHandle_seActualizaConCadaCambioDelFormulario() {
        val context = ApplicationProvider.getApplicationContext<Context>()
        val handle = SavedStateHandle()
        val viewModel = EcoTripViewModel(handle, EcoTripDataStore(context))

        viewModel.onDestinoChange("Cuenca")
        viewModel.onDiasChange("3")
        viewModel.onModoGrupalChange(true)

        assertEquals("Cuenca", handle.get<String>(EcoTripViewModel.KEY_DESTINO))
        assertEquals("3", handle.get<String>(EcoTripViewModel.KEY_DIAS))
        assertEquals(true, handle.get<Boolean>(EcoTripViewModel.KEY_ES_GRUPAL))
    }

    @Test
    fun validacion_diasInvalidosGeneranError() {
        val context = ApplicationProvider.getApplicationContext<Context>()
        val viewModel = EcoTripViewModel(SavedStateHandle(), EcoTripDataStore(context))
```

```
        viewModel.onDiasChange("abc")
        assertNotNull(viewModel.uiState.value.errorDias)

        viewModel.onDiasChange("0")
        assertNotNull(viewModel.uiState.value.errorDias)

        viewModel.onDiasChange("")
        assertNotNull(viewModel.uiState.value.errorDias)

        viewModel.onDiasChange("5")
        assertNull(viewModel.uiState.value.errorDias)
    }

    @Test
    fun validacion_destinoVacioGeneraError() {
        val context = ApplicationProvider.getApplicationContext<Context>()
        val viewModel = EcoTripViewModel(SavedStateHandle(), EcoTripDataStore(context))

        viewModel.onDestinoChange("   ")
        assertNotNull(viewModel.uiState.value.errorDestino)

        viewModel.onDestinoChange("Loja")
        assertNull(viewModel.uiState.value.errorDestino)
        assertFalse(viewModel.uiState.value.destino.isBlank())
    }
}
```